

TABLES

Paper Title: Semantic-Structural Microservices Decomposition: Unifying Graph Neural Networks and Large Language Models for Multi-Objective Software Architecture Transformation

Authors: Dr. Jay Nanavati, Dr. Sanskruti Patel, Dr. Unnati Patel

Conference: 6th International Conference on Smart Electronics and Communication - ICOSEC 2025

Table I. Positioning of the Proposed Hybrid GNN + LLM Framework Against Existing Microservice Decomposition Methods

Title / Method	Multi-Objective Optimization	Graph + Semantic Fusion	GNN-Based Structure	LLM-Based Semantics	Year
Outlier Refactoring via [10]	×	×	GNN	×	2021
Monolith to Microservices via HetGNN [11]	×	×	Hetero GNN	×	2022
Static/Dynamic Feature-Aware GNN [12]	×	×	VAE + GNN	×	2023
Directed GAT for Software Migration [13]	×	×	GAT	×	2024
MonoEmbed: Contrastive LLM Embeddings [14]	×	×	×	LLM-only	2025
<i>The proposed work (Hybrid GNN + LLM)</i>	✓	✓	✓	Code-aware LLM	2025

Table II. Type of Edges in Multigraph

Type of edges	What do they represent?
EXTENDS/IMPLEMENTNS	Inheritance/implementation hierarchies
CALLS	Direct method invocations
OVERRIDES	Method redefinitions across class hierarchies
ACCESS_FIELD	Attribute-level coupling

Table III. Toolchain for Implementing the Proposed Methodology on Petclinic

Stage-I Structural Decomposition and Graph Construction			
Step	Tool / Library	Input (from PetClinic)	Output
1.1 Java Analysis	<u>Soot</u>	Java source code (e.g., OwnerController, Pet, VisitRepository)	Control Flow Graphs (CFGs) Call Graphs (CGs) Class Hierarchy Graphs (CHGs)
1.2 Graph Construction	<u>JGraphT</u>	CFGs, CGs, CHGs (from Soot)	Heterogeneous directed multigraph $G = (V, E)$ (nodes = classes/methods; edges = CALLS, EXTENDS, IMPLEMENTS, etc.)
1.3 Graph Serialization	Jackson JSON Processor [23]	JGraphT multigraph	petclinic_graph.json / .csv (nodes + edges annotated with properties)
Stage-II Representation Learning and Semantic Refinement			
Step	Tool / Library	Input (from PetClinic)	Output
2.1 Graph Embedding	PyTorch Geometric [24]	Serialized graph (petclinic_graph.json)	Node embeddings (64-dim vectors per class/method)
2.2 Clustering	<u>NetworkX</u> [25] and <u>python-louvain</u> [26]	Node embeddings	Candidate microservice clusters (e.g., Owner, Pet, Visit groups)
2.3 Semantic Refinement	OpenAI GPT-4 [†] [27]	Candidate clusters + code comments/docs	Refined microservice modules with meaningful names + natural-language rationales

† GPT-4 was employed due to its superior performance in semantic reasoning and contextual code understanding, both of which are critical for refining candidate microservices. Alternative LLMs such as Code Llama [28] and StarCoder [29] are competitive in code synthesis, but remain less validated for higher-level semantic alignment tasks in microservice decomposition.

Table IV. Comparative Positioning of R-GCN against Alternative Embedding Methods

Method	Edge-Type Handling	Node/Edge Heterogeneity	Scalability	Suitability for Monolith Decomposition
HetGNN [30]	Heterogeneous	Multiple types	Low–Moderate	Strong expressiveness but impractical for mid-scale systems
GAT [31]	Only Homogeneous	Single type	Moderate	Limited: Requires large data; cannot differentiate dependency types
Node2Vec [32]	Only Homogeneous	Single type	High	Limited: Ignores semantic diversity in software graphs
R-GCN [33]	Multi-relational support	Limited Node types limited but fully modeled edge types	Moderate	Best trade-off: Captures CALLS/EXTENDS/IMPLEMENTS relations with scalability

Table V. Comparative Positioning of Louvain Modularity Optimization Against Alternative Clustering Methods

Method	Graph Awareness	Input Parameters	Scalability	Suitability for Microservice Extraction
Hierarchical Clustering [34]	Distance-based	Linkage criterion	Low	Produces dendrograms, but not scalable for larger codebases
Spectral Clustering	Uses graph Laplacian	Requires number of clusters k	Moderate	Effective for balanced partitions, but parameter sensitive
K-Means [35]	No (vector space only)	Requires number of clusters k	High	Limited: ignores graph topology, may split cohesive modules
Louvain Modularity	Community detection	No preset k required	High	Best suited: maximizes modularity, discovers natural service boundaries

Table VI. Hyper-parameters Used in the Proposed Methodology

Hyperparameter	Value	Stage
Embedding dimension	64	Representation Learning (R-GCN)
Number of layers	2	Representation Learning (R-GCN)
Learning rate	0.01	Representation Learning (R-GCN)
λ (Eq. 2 trade-off parameter)	0.5	Representation Learning (R-GCN)
Resolution parameter	1.0 (default)	Clustering (Louvain)
Model	GPT-4 (gpt-4-turbo)	Semantic Refinement (GPT-4)

Table VII. Metrics used during Different Stages of The Methodology

Stage	Process	Output Artifact	Metric Used
I	Graph Construction	Dependency Graph	Directed Density [39] Average Out-degree [39]
II	Graph Embedding using R-GCN	Node Embeddings (64-dim)	Silhouette Score [40]
	Louvain Clustering	Service Candidate Partitions	Modularity Q [41] Cohesion, Coupling [42]
	Semantic Refinement using GPT-4	Refined Candidates and Labels	NMI [43], SC [44], CLS [45]

Table VIII. Effect of Semantic Refinement using GPT-4 on Semantic Quality

Metric	Before GPT-4-enabled Refinement	After GPT-4-enabled Refinement
NMI	0.0	0.41
SC	0.34	0.57
CLS	0.42	0.71

Table IX. Louvain Clustering Results on Petclinic

Cluster ID	Assigned Members	Cohesion	Coupling
Cluster 0	{Owner, OwnerController, OwnerRepository, Person}	1.0000	0.7500
Cluster 1	{Pet, PetController, PetRepository}	0.6667	1.0000
Cluster 2	{Visit, VisitController, VisitRepository}	0.6667	1.0000
Cluster 3	{Vet, VetController, VetRepository, Specialty}	1.0000	0.7500
Cluster 4	{BaseEntity, NamedEntity}	1.0000	0.5000
Cluster 5	{AbstractJpaRepository, JpaRepository}	1.0000	0.5000
Cluster 6	{PetType, PetTypeRepository}	1.0000	0.5000
Cluster 7	{AbstractController}	0.0000	0.0000
Cluster 8	{AbstractEntity}	0.0000	0.0000
Cluster 9	{AbstractService}	0.0000	0.0000